Integration of the Reichardt and Duprat wall models into Neko (`feature/wmles`)

Work report

Prepared with Claude Code for M. Münsch

12 October 2026, extended 27 October 2026

Abstract

This report documents the port of the Reichardt and the Duprat et al. wall models from the `curved_neko` fork (Master's thesis of D. Bole) into the `feature/wmles` branch of `muenschm/neko`, between 29 September and 8 October 2026. The work was done in six pull requests, all merged. Both models are now available on the CPU backend and, partially untested, on the CUDA, HIP, OpenCL and Metal backends. The review of the original implementation against the paper of Duprat et al. (2011) found several issues, most importantly a spurious Jacobian factor in the pressure gradient ("FIX 10") that reduced the gradient seen by the model by a factor of roughly 250–2700 on the periodic-hill mesh, and an inaccurate quadrature of the velocity profile. The report records the findings, the decisions taken, the implementation, the tests with their exact settings, and the open items. The extension of 27 October covers a mesh-connectivity bug that stopped Intel builds on more than one rank (fixed in PR #7), and the first periodic-hill results at $Re_h = 10\,595$ with the Reichardt model on CPU (Fritz) and GPU (Alex), compared with the LES database of Gloerfelt & Cinnella (2019). The recirculation zone comes out much too short: the flow separates late and reattaches at $x/h \approx 2.7$–2.8 instead of 4.2–4.5. GPU and CPU agree in the mean flow, but the shear-layer turbulence is 5–13 % higher on the GPU, which a deterministic test should clarify.

Contents

1 Summary **2**

2 Starting point **3**
2.1 Repositories . . . 3
2.2 API drift and porting strategy . . . 3

3 Workflow and pull requests **3**

4 Reichardt wall model **4**
4.1 Formulation . . . 4
4.2 CPU port (PR #1) . . . 4
4.3 GPU kernels (PR #2) . . . 4
4.4 Configurable constants (PR #3) . . . 4
4.5 Verification . . . 4

5 Duprat wall model **5**
5.1 Formulation . . . 5
5.2 Review of the original implementation . . . 5
5.3 The Jacobian factor ("FIX 10") . . . 6
5.4 Quadrature of U^* . . . 7
5.5 Constant pressure gradient on CPU (PR #4) . . . 7

5.6 Local pressure gradient (PR #5) . . . 8
5.7 GPU kernels (PR #6) . . . 9

6 Case-file conventions **10**

7 Wall-normal velocity (penetration) check **10**

8 Multi-rank failure on Fritz: `Invalid face neigh.` (PR #7) **11**

9 Periodic hill at $Re_h = 10\,595$ with the Reichardt model **12**
9.1 Case and runs . . . 12
9.2 Evaluation . . . 12
9.3 Forces and momentum balance . . . 13
9.4 Skin friction, separation and reattachment . . . 13
9.5 Profiles . . . 15
9.6 Discussion . . . 15
9.7 GPU (Alex) versus CPU (Fritz) . . . 18
9.8 Performance . . . 18

10 Periodic hill at $Re_h = 10\,595$ with the Duprat model **19**
10.1 Case and runs . . . 19
10.2 Forces . . . 19
10.3 Skin friction, separation and reattachment . . . 20
10.4 Profiles . . . 21
10.5 The pressure gradient seen by the model . . . 22
10.6 GPU (Alex) versus CPU (Fritz) . . . 22
10.7 Performance . . . 25
10.8 Discussion . . . 25

11 Test environment **25**

12 Open items and recommendations **26**

A Case-file examples **27**

B Mapping of the student's case-file keys **27**

C Files added or changed **28**

1 Summary

- **Reichardt** (`"model": "reichardt"`): CPU (PR #1), CUDA/HIP/OpenCL/Metal (PR #2), configurable constants `C`, `B1`, `B2` (PR #3).
- **Duprat** (`"model": "duprat"`): CPU with a constant prescribed pressure gradient (PR #4), locally evaluated and filtered pressure gradient (PR #5), CUDA/HIP/OpenCL/Metal for both modes (PR #6).
- The Duprat port follows the paper [1] rather than the original code. Five issues of the original implementation were corrected (Section 5.2); two of them, the Jacobian factor and the quadrature, may affect the results reported in the thesis [5].
- **GPU code:** the device code of both models was checked as follows.
  - The device code was first verified on the host and, for OpenCL, by running Neko's full solver through PoCL.
  - Since then, both models have run the full periodic hill on one NVIDIA GPU (Alex, CUDA).
  - HIP and Metal have not run on hardware.
  - The rank-0 warning that the GPU implementation is partially untested has been removed from both models on the upstream branches.
- No spurious wall-normal velocity was found, on flat or curved walls (Section 7).
- A mesh-connectivity bug made Intel builds fail on more than one rank (`Invalid face neigh.`); it was fixed in PR #7 and is merged upstream as ExtremeFLOW/neko#2867 (Section 8).
- **Periodic hill, Reichardt** (Section 9).
  - *Mean flow:* the outer flow and the recovery region agree well with Gloerfelt & Cinnella [3].
  - *Bubble:* the flow separates late ($x/h \approx 0.55$) and reattaches early ($x/h \approx 2.7$–2.8, reference 4.2–4.5). This is consistent with an equilibrium wall model that ignores the adverse pressure gradient at the crest, together with a coarse wall-normal resolution there.
  - *GPU versus CPU:* the runs agree in the mean flow, the skin friction and the separation point. The shear-layer turbulence was higher on the GPU. The Duprat runs, made with the same builds, do not show this, so it is probably not a code-path difference. A deterministic restart test is prepared to settle it.
- **Periodic hill, Duprat** (Section 10).
  - *Improvements over Reichardt:* the windward skin-friction peak now matches the reference (0.031 against 0.033). The flow separates earlier (0.45) and reattaches later (3.2–3.4).
  - *Remaining deficits:* the reattachment is still about 1 short of the reference. Drag and skin friction are 33–43 % higher than with Reichardt.
  - *Cause:* in the bubble and in the recovery region the local mode passes on only about a tenth of the mean pressure gradient. Three improvements of the local mode are proposed.
  - *GPU versus CPU:* no systematic difference.

- **Upstream:** both models are prepared as separate pull requests against ExtremeFLOW/neko `develop`, each with a pFUnit suite (Section 12).

Open items are listed in Section 12.

2 Starting point

2.1 Repositories

`muenschm/neko`, branch `feature/wmles` in sync with Neko `develop`; head at the start: `3837897` ("Update joblimit check ... (#2817)").

`muenschm/curved_neko`, branch `feature/bc` the student's work as a single commit `f2b81c1` ("Initial commit", D. Bole, 26 May 2026) on top of an older Neko `develop` (`a4aaf2c`). The commit contains 16 files (+3316/−10 lines):

- the models: `reichardt.f90`, `reichardt_cpu.f90`, `duprat.f90`, `duprat_cpu.f90`, plus changes to `wall_model_fctry.f90` and `Makefile.am`;
- build artefacts (`build_curved/`: `libneko.lai`, generated `neko.h`, `intf.py`);
- a periodic-hill example including the 1.6 MB mesh twice (`examples/periodic hill/` and its `run1/` copy).

2.2 API drift and porting strategy

The wall-model API of Neko changed considerably between the student's base and `feature/wmles`, mainly through upstream PR #2646 ("Wall model sampling enhancements", T. Mukha, 20 August 2026):

Student's base	`feature/wmles`
`init(..., h_index, json)`	`init(..., json)`; sampler from `wall_sampler_factory`
`ind_r/s/t/e`, `h` in `wall_model_t`	`class(wall_sampler_t) :: sampler`, `sampler%h`
`partial_init(coef, json)`	`partial_init(coef, scheme_name, json)`
`finalize(msk, facet)`	`finalize(msk, facet, bc_name, user)` + `validate_single_sample()`
CPU kernels index full $l_x^3 n_{el}$ fields	kernels work on pre-sampled vectors (`sampler%sample`)

A merge or cherry-pick was therefore not possible. The models were ported by hand, step by step, one pull request per testable step, with `spalding` and `cai_sagaut_model_ii` as templates.

3 Workflow and pull requests

All work was done on the branch `claude/stoic-euler-8zyrhx` of `muenschm/neko`. After each merge, the branch was reset to the new `feature/wmles`, so each PR contains exactly one step. The first push failed (HTTP 403) until GitHub was reconnected for the session. All PRs were reviewed and merged by M. Münsch.

PR	Title	Head	Merge commit	Merged
#1	Add the Reichardt wall model (CPU backend)	`b0b064e`	`e95465a`	30 Sep
#2	Add device backends for the Reichardt wall model	`382d1a9`	`29ff2d1`	7 Oct
#3	Make the Reichardt constants configurable	`a23da72`	`d7978b5`	8 Oct
#4	Add the Duprat wall model with a constant pressure gradient (CPU)	`62b734a`	`f419a96`	8 Oct
#5	Duprat wall model: local pressure-gradient mode	`25155a4`	`ff1e65d`	8 Oct
#6	Add device backends for the Duprat wall model	`1b5fbc0`	`b70f8d7`	8 Oct

In total, 31 files changed (+5254/−3 lines) between `3837897` and `b70f8d7`; see Appendix C.

4 Reichardt wall model

4.1 Formulation

Reichardt's law of the wall [2],

$$u^+ = \frac{1}{\kappa}\ln(1+\kappa y^+) + C\left[1 - e^{-y^+/B_1} - \frac{y^+}{B_1}\,e^{-y^+/B_2}\right], \qquad C = 7.8,\ B_1 = 11,\ B_2 = 3, \quad (1)$$

is solved for u_τ from $f(u_\tau) = u_\tau\, u^+(hu_\tau/\nu) - U = 0$ with Newton's method and the analytical derivative $f' = u^+ + y^+\,\mathrm{d}u^+/\mathrm{d}y^+$. Here U is the magnitude of the sampled wall-parallel velocity. The wall stress is $\boldsymbol{\tau}_w = -\rho_w u_\tau^2\,\mathbf{u}_t/|\mathbf{u}_t|$.

4.2 CPU port (PR #1)

The port follows `spalding`. Differences to the original code:

- The wall stress is scaled by the wall density ρ_w, consistent with the other wall models. This changes nothing for $\rho = 1$.
- The parameter `B` was accepted but never used, so it was removed.
- OpenMP in the kernel, a zero-velocity guard, and a precision-aware Newton tolerance $\max(10^{-8}, 10\,\varepsilon_{\mathrm{mach}})$; with 10^{-8} alone, single precision would not converge.

4.3 GPU kernels (PR #2)

CUDA and HIP (templated on the precision), OpenCL and Metal (FP32) kernels, structured like `spalding` and `cai_sagaut_model_ii`. At M. Münsch's request, the device backends print a rank-0 warning ("The reichardt wall model GPU implementation is partially untested so far") instead of stopping with an error. After the hill runs on Alex (Sections 9.7 and 10.6), this warning was removed again on the upstream branches of both models.

4.4 Configurable constants (PR #3)

To treat model constants consistently across wall models (Section 6), C, B_1, B_2 became optional case-file parameters `C`, `B1`, `B2`, with the defaults of (1). The names `B1`/`B2` were chosen by M. Münsch, to avoid confusion with coordinate directions (instead of the proposed `y1`/`y2`).

4.5 Verification

- **Kernels, known answer.** Sampled velocities constructed from a known u_τ for $y^+ = 0.5$–500 and $\rho_w \in \{1, 1.2\}$; the kernels recover u_τ to 8.7×10^{-16} (double) and 7.5×10^{-8} (single; 2.4×10^{-7} for OpenCL). This was tested for:

– the CPU kernel;
– CUDA and HIP, compiled as host C++ with stubbed keywords;
– Metal, stubbed;
– OpenCL, the generated source JIT-compiled through PoCL.

The wall-normal velocity is removed, and zero velocity gives zero stress. After PR #3 the same tests also passed with a non-default set $(C, B_1, B_2) = (6, 9, 2.5)$.

- **Full solver.** `turb_channel` example, $N = 3$, 20 steps, $\Delta t = 0.01$; wall-model BC on zones 3 and 4, GLL sampling at index 2; force on the bottom wall:

Case	F_x
`spalding`	0.42830×10^{-2}
`reichardt` (CPU, unchanged after #2)	0.42879×10^{-2}
`reichardt`, explicit default constants (#3)	0.42879×10^{-2}
`reichardt`, $(C, B_1, B_2) = (6, 9, 2.5)$	0.44443×10^{-2}

- `lint.sh` 10/10; `validate_case_schema.py` accepts valid cases and rejects a case without `kappa`.

5 Duprat wall model

5.1 Formulation

Duprat et al. [1] integrate the simplified thin-boundary-layer equation across the wall layer, using an extended velocity scale. With $\partial p/\partial s$ the wall-tangential pressure gradient along the local flow direction:

$$u_p = \left|\frac{\nu}{\rho}\frac{\partial p}{\partial s}\right|^{1/3}, \qquad u_{\tau p} = \sqrt{u_\tau^2 + u_p^2}, \qquad \alpha = \frac{u_\tau^2}{u_{\tau p}^2}, \qquad y^* = \frac{y\,u_{\tau p}}{\nu}, \tag{2}$$

$$\frac{\mathrm{d}U^*}{\mathrm{d}y^*} = \frac{s_p\,(1-\alpha)^{3/2}\,y^* + \alpha}{1 + \nu_t/\nu}, \tag{3}$$

$$\frac{\nu_t}{\nu} = \kappa\, y^* \left[\alpha + y^*(1-\alpha)^{3/2}\right]^{\beta} \left(1 - e^{-y^*/(1+A\alpha^3)}\right)^2, \qquad \beta = 0.78,\ A = 17. \tag{4}$$

Here s_p is the sign of the pressure gradient relative to the flow (adverse positive). Equations (4), (5) and (6) of the paper correspond to (2), (3) and (4). The paper writes the pressure velocity as $u_p = |(\mu/\rho^2)\,\partial P/\partial x|^{1/3}$, which is identical, since Neko's `p` is the dynamic pressure: the pressure equation uses $h_1 = 1/\rho$. The friction velocity follows from $f(u_\tau) = u_{\tau p}\,U^*(h u_{\tau p}/\nu) - U = 0$. For $\partial p/\partial s = 0$, $\alpha = 1$ and the model reduces to its equilibrium form.

5.2 Review of the original implementation

The original code (`curved_neko`) was compared with the paper, with the thesis report [5], and with Neko's operators.

Item	Original code	Ported version
Pressure gradient ("FIX 10") u_p	`dudxyz`$(p)\times$`coef%jac`, assuming `dudxyz` returns $(1/J)\,\partial p/\partial x$ $(\nu\lvert\mathrm{d}p/\mathrm{d}x\rvert/2)^{1/3}$	`grad`(p), the physical gradient (Section 5.3) $(\nu\lvert\partial p/\partial s\rvert/\rho_w)^{1/3}$, Eq. (2); also the form given in the report

Item	Original code	Ported version
Eddy viscosity	$(\kappa + \beta(1-\alpha)^{3/2})\, y^*(1-e^{\cdots})^2$	Eq. (4); the report's listing also shows this form
Wall-stress term in (3)	$+1$	$+\alpha$
U^* quadrature	one 5-point Gauss rule on $[0, y^*]$	composite 12×5 Gauss rule in $\ln(1+y^*)$ (Section 5.4)
Solver	Newton; can collapse to $u_\tau \approx 0$ for favourable gradients	root bracketed by sign change, Newton with bisection fallback
Gradient time level	∇p^{n-1} (double buffer) with $\mathbf{u}^n$	∇p^n, the same time level as $\mathbf{u}^n$ (Section 5.6)
Streamwise gradient	$\lvert\nabla_t p\rvert \cdot \operatorname{sign}(\nabla_t p \cdot \mathbf{u}_t)$	projection $\nabla p \cdot \mathbf{u}_t / \lvert\mathbf{u}_t\rvert$
Stokes clamp	$\lvert\partial p/\partial s\rvert \le 2(u\nu/h)^{3/2}/\nu$	$\lvert\partial p/\partial s\rvert \le (\rho_w/\nu)(u\nu/h)^{3/2}$, consistent with u_p
Filter	$\epsilon = \min(\Delta t/T, 0.5) \cdot$ ramp, pre-activation buffer	$\epsilon = 1 - e^{-\Delta t/T}$, filtered value starting from zero

The eddy-viscosity and $+1/+\alpha$ differences, as well as the factor in u_p, vanish for $\alpha = 1$. They therefore do not affect the zero-pressure-gradient channel runs of the thesis. The temporal filter, ramp and clamp are not part of the paper, which evaluates τ_w locally from the filtered LES velocity in the first cell. They are numerical stabilisation added by the student, and are kept as documented, adjustable safeguards.

5.3 The Jacobian factor ("FIX 10")

The original code multiplied the output of `dudxyz` by `coef%jac`. Its comment states that `dudxyz` returns $(1/J)\,\partial p/\partial x$ and that J "varies by factors 2–5". Both assumptions are incorrect:

- `coef%drdx` etc. are cofactors, i.e. $J\,\partial r/\partial x$ (`sem/coef.f90`, around line 1146). `dudxyz` multiplies by `jacinv`, so its result is the physical gradient. Neko's own `curl` uses it that way (`opr_cpu.f90`, lines 150–165).
- J is not a ratio of order one but the volume scaling of the map from the reference element $[-1,1]^3$, i.e. about 1/8 of the element volume.

Test. On the student's periodic-hill mesh (`box.nmsh`, 6688 elements, his mesh deformation, $N = 7$), $p = x + 2y - 3z$ was set and `dudxyz` applied:

	$\partial p/\partial x$	$\partial p/\partial y$	$\partial p/\partial z$
`dudxyz`	1.0000 (everywhere)	2.0000	-3.0000
`dudxyz`×`jac` (FIX 10)	3.7×10^{-4} – 4.1×10^{-3}	7.4×10^{-4} – 8.2×10^{-3}	–

Consequence. With FIX 10 the model sees a pressure gradient that is 250–2700 times too small. Since $u_p \propto \lvert\partial p/\partial s\rvert^{1/3}$, u_p is 6–14 times too small and α stays close to 1, so the pressure-gradient correction is nearly switched off. This agrees with three observations:

- the code comment of FIX 10, which reports a CFL blow-up at $t = 110.5$ after the pressure gradient was switched on at $t_{\text{filter,start}} = 100$; FIX 10 "fixed" it;
- the thesis conclusion that Duprat showed no systematic advantage over Reichardt on the periodic hill (Sections 6.1–6.2 of [5]);
- the thesis report itself (Section 3.1.3) states correctly that `dudxyz` gives the physical gradient, and does not mention FIX 10.

Caveat. The final committed code contains FIX 10, but which code version produced the production runs of the thesis could not be established from the documents. This should be confirmed with the student.

Decision. FIX 10 was dropped. The gradient is computed with Neko's `grad` (three `dudxyz` calls, with a device variant).

5.4 Quadrature of U^*

$U^*(y^*)$ was compared against SciPy's adaptive quadrature. As a check of the formulation itself, the reference gives $U^*(1000) = 22.379$ at $\alpha = 1$, consistent with Fig. 3 of [1] (≈ 22.5).

Quadrature	worst rel. error in U^*	comment
single 5-point Gauss rule on $[0, y^*]$ (original)	up to 60 %	48–60 % at $y^* \geq 10^3$, $\alpha = 1$
composite 8×5 Gauss in $\ln(1 + y^*)$	8.6×10^{-7}	
composite 12×5 Gauss in $\ln(1 + y^*)$ (ported)	5.8×10^{-9}	

Tested for $\alpha \in \{1, 0.7, 0.3, 0.05\}$, adverse and favourable gradients, and y^* up to 10^4. At $\alpha = 1$ and the sampling heights of the thesis channel runs, the original rule gives:

y^*	exact U^*	original rule
33.3	13.945	13.945 (−0.0 %)
60	15.568	15.421 (−0.9 %)
153	17.832	19.308 (+8.3 %)

The thesis reports an 8.4 % DNS mismatch of Duprat at `h_index` $= 7$ ($y^+ \approx 153$) and attributes it to the model's velocity profile. The close agreement with the +8.3 % quadrature error suggests that this deterioration is, at least partly, a quadrature artefact.

5.5 Constant pressure gradient on CPU (PR #4)

- `kappa` is required; `beta` (0.78) and `A` (17) are optional.
- The pressure gradient is set by the optional object `"pressure_gradient": {"type": "constant", "value": ...}`; without it, the gradient is zero (equilibrium limit). `value` is $\partial p/\partial s$ along the local flow direction, positive if adverse. It replaces the student's pair `use_constant_dpdx: true` / `dpdx_constant` (Appendix B).
- The residual is not monotone in u_τ for favourable gradients: it first decreases, then increases. A root always exists ($f(0) < 0$, $f \to \infty$), but plain Newton walks into the decreasing branch and gives $u_\tau \approx 0$. The solver therefore first brackets the root by a sign change and replaces Newton steps that leave the bracket by bisection.

Verification.

- 96 known-answer cases (velocity from the SciPy integral for a known u_τ): $y^+ \in [1, 2000]$, $u_p/u_\tau \in \{0, 0.3, 1, 2, 5\}$, adverse and favourable, $\rho \in \{1, 1.3\}$. Maximum relative error of u_τ: 1.2×10^{-7} (double) and 1.4×10^{-5} (single). Cases where the model gives $U < 0$ (strong favourable gradient) were excluded, since a velocity magnitude cannot be negative.
- 48 strong favourable-gradient cases: a true root is found in every case, $|f|/U < 7 \times 10^{-14}$, where the plain Newton solver gave $u_\tau \approx 0$.

- `turb_channel`, $N = 3$, 20 steps, GLL sampling index 2, bottom wall force:

Case	F_x
`reichardt`	0.42879×10^{-2}
`duprat`, zero gradient	0.42872×10^{-2}
`duprat`, $\partial p/\partial s = -0.0036$ ($\approx -u_\tau^2/\delta$)	0.43835×10^{-2} (+2.2 %)
`duprat`, `"type": "local"` (before PR #5)	rejected with an error

  The favourable gradient flattens the near-wall profile, so the same sampled velocity implies a higher wall stress.

- The same runs with the legacy `"h_index": 2` instead of `"sampling": {"type": "gll", "value": 2}` give identical forces (Section 6).

5.6 Local pressure gradient (PR #5)

Timing. In `fluid_pnpn_step`, the wall stress is computed once per time step, when the Neumann contributions are added to the right-hand side (`fluid_pnpn.f90`, line 758). This happens *before* the pressure and velocity solves. When τ_w for step $n+1$ is computed, the newest fields are therefore $\mathbf{u}^n$ and p^n. The original code used ∇p^{n-1} with $\mathbf{u}^n$. As suggested by M. Münsch, the ported version uses ∇p^n, the same time level as the sampled velocity. Using p^{n+1} is not possible without restructuring the solver.

Per time step.

1. ∇p with `grad` (physical gradient, no Jacobian factor);
2. sampling of the three components through the model's `sampling`, at the same points as the velocity;
3. projection on the local flow direction, $\partial p/\partial s = \nabla p \cdot \mathbf{u}_t/|\mathbf{u}_t|$ (the paper's streamwise gradient; the magnitude of the full tangential gradient would also pick up spanwise fluctuations);
4. Stokes limit $|\partial p/\partial s| \le (\rho_w/\nu)(u\nu/h)^{3/2}$, i.e. $u_p \le \sqrt{u\nu/h}$;
5. exponential moving average, as decided by M. Münsch:

$$(\partial p/\partial s)^n_{\text{filt}} = (1-\epsilon)\,(\partial p/\partial s)^{n-1}_{\text{filt}} + \epsilon\,(\partial p/\partial s)^n\,, \qquad \epsilon = 1 - e^{-\Delta t/T}. \tag{5}$$

   This weight is the exact discrete form of a first-order low-pass filter for variable Δt; $T = 0$ gives $\epsilon = 1$ (no filtering).

The update is done once per time step. Before `start_time` the filtered gradient stays zero. Since it starts from zero, it enters over about one filter time; the student's separate ramp and pre-activation buffer are therefore not needed.

Case file. `"pressure_gradient": {"type": "local", "filter_time": T, "start_time": t0}`. `filter_time` is required, because it is dimensional and no default is meaningful; `start_time` defaults to 0.

Diagnostics. The filtered $\partial p/\partial s$ and α at the wall nodes are stored in the registry fields `duprat_dpds` and `duprat_alpha`, like `tau`, and can be written with `field_writer`.

Limitation. The filter state is not checkpointed. After a restart it starts again from zero.

Verification.

- **Update kernel, exact checks:**
  - projection, including removal of the wall-normal velocity and gradient components;
  - sign for forward and reversed flow;
  - clamp equal to the Stokes bound (31.62 in the test);
  - a node without flow keeps its previous value;
  - the filter reproduces $G(1 - e^{-t/T})$ for constant input to 6×10^{-14} over 200 steps.
- **Known answer in a full run.** A user `preprocess` hook overwrote $p = G(x \cos\varphi + z \sin\varphi)$ before each step ($G = 0.05$, $\varphi = 60°$), in uniform flow along x, so the expected $\partial p/\partial s = 0.025$ at all 10 368 wall nodes:
  - without filter: 0.02500 from the second step on, maximum deviation 2.5×10^{-6};
  - $T = 0.05$, $t_0 = 0.035$: zero before t_0, then (5) step by step, maximum deviation 1.3×10^{-6}.
- **Turbulent channel**, $N = 3$, 200 steps, $T = 1$:
  - stable;
  - $F_x = 0.37652 \times 10^{-2}$ vs 0.37222×10^{-2} with zero gradient (+1.2 %);
  - filtered $\partial p/\partial s$ about ± 0.03–0.04, mean -4×10^{-4}, as expected in a body-force-driven channel;
  - α mostly > 0.6; one node reached $\alpha \approx 0$ transiently at step 60 (the model's local separation limit) and recovered.
- The schema rejects `local` without `filter_time`, and filter settings combined with `constant`.

5.7 GPU kernels (PR #6)

Two kernels per backend, `duprat_compute` and `duprat_update_dpds`, mirroring the CPU kernels; plus device paths in `duprat.f90`:

- gathering ν and ρ at the wall;
- `grad` on device pointers;
- vector fills;
- the masked scatter into the diagnostic fields.

The CPU-only error was replaced by the rank-0 "partially untested" warning.

Robustness fix (CPU and GPU). α is formed as $u_\tau^2/(u_\tau^2 + u_p^2)$ instead of $u_\tau^2/u_{\tau p}^2$, and $1 - \alpha$ is limited to ≥ 0 before the power 3/2. With relaxed GPU math, e.g. Metal fast math, rounding could otherwise give α slightly above 1 and NaN. On IEEE hardware the results are unchanged; no case with $\alpha > 1$ occurred in 2×10^5 random samples in either precision.

Verification.

Kernel	known u_τ: rel. error u_τ (α)	favourable cases	update kernel
CPU (reference)	1.2×10^{-7} (8.8×10^{-9})	root found	exact
CUDA/HIP, host C++, double	1.2×10^{-7} (8.8×10^{-9})	$\lvert f\rvert/U \le 10^{-14}$	3×10^{-16}
CUDA/HIP, host C++, float	1.4×10^{-5} (1.0×10^{-6})	$\le 4 \times 10^{-6}$	8×10^{-8}
Metal (stubbed), float	1.4×10^{-5} (1.0×10^{-6})	$\le 4 \times 10^{-6}$	8×10^{-8}
OpenCL via PoCL, double	1.2×10^{-7} (8.8×10^{-9})	finite, non-zero	3×10^{-16}
OpenCL via PoCL, float	1.0×10^{-5} (7.4×10^{-7})	finite, non-zero	8×10^{-8}

Full solver on the OpenCL backend. PoCL does not implement `clEnqueueWaitForEvents`. For this test only, a preloaded shim replaced it by the blocking `clWaitForEvents`; Neko's sources were not changed. The turbulent channel ($N = 3$, 30 steps, one rank) gave:

Case	F_x CPU	F_x OpenCL	$\partial p/\partial s$, α at the wall
zero gradient	0.43246×10^{-2}	0.43246×10^{-2}	identical
constant $\partial p/\partial s = -0.0036$	0.44200×10^{-2}	0.44200×10^{-2}	identical
`local`, $T = 0.05$	0.45788×10^{-2}	0.45788×10^{-2}	identical

In the `local` case, $\partial p/\partial s \in [-0.187, 0.198]$ and $\alpha \in [0.116, 1]$. Every reported value above 10^{-6} is identical at every step; only the spanwise force ($\sim 10^{-10}$) differs, by at most 10^{-12}.

6 Case-file conventions

Model constants. The existing models follow two conventions:

Model	`kappa`	further constants
`spalding`	required	`B` required
`cai_sagaut_model_ii`	required	`B` required; `p`, `s` optional (paper defaults)
`rough_log_law`, `most`, `richardson`	optional (0.4)	`B` optional (0)
`reichardt` (new)	required	`C`, `B1`, `B2` optional (7.8, 11, 3)
`duprat` (new)	required	`beta`, `A` optional (0.78, 17)

The new smooth-wall models follow `cai_sagaut_model_ii`: log-law constants are required, model-specific calibration constants are optional with paper defaults, and no constant is hardcoded. The atmospheric-boundary-layer models were left unchanged, since changing them would break existing case files.

Sampling. `sampling` is the documented successor of `h_index` (upstream PR #2646: "the now legacy `h_index` remains supported"). All wall models go through the shared `wall_sampler_factory`, which uses `sampling` if present and falls back to `h_index` otherwise; this was confirmed by identical results. The new documentation uses `sampling`, as decided by M. Münsch. `sampling` also allows `"type": "distance"`, and in the `local` mode the pressure gradient goes through the same sampler as the velocity.

7 Wall-normal velocity (penetration) check

The wall-model boundary condition is a `shear_stress` condition of type `BC_MIXED_CONSTRAINS_NORMAL`. With a coupled velocity solver (e.g. `coupled_cg`), Neko's `vector_bc_projector` builds a local basis $(\mathbf{n}, \mathbf{t}_1, \mathbf{t}_2)$ at each wall node and constrains the normal component strongly. At element edges and corners it uses an averaged node normal (face normals summed, gather-scatter, normalised). The wall model only sets the tangential stress.

Measure of $\mathbf{u} \cdot \mathbf{n}$	Duprat `local`, $T = 0$	Reichardt
Channel, all 10 368 wall nodes, 100 steps	$\leq 1.3 \times 10^{-15}$	–
Hill, curved wall, facet-interior nodes	$\leq 4 \times 10^{-16}$	$\leq 7 \times 10^{-16}$
Hill, curved wall, averaged node normal, all nodes	$\leq 6 \times 10^{-16}$	$\leq 7 \times 10^{-16}$
Hill, curved wall, per-element normals, all nodes	≈ 0.02 max, 0.003 RMS	same
Hill, net flux $\int \mathbf{u} \cdot \mathbf{n} \, \mathrm{d}A/A$	$\sim 3 \times 10^{-6}$	$\sim 5 \times 10^{-6}$

Settings for the hill: the student's mesh at $N = 4$, uniform initial flow along x (which initially crosses the curved wall), GLL sampling index 2. The per-element measure was run up to $t = 1$ (142 steps for Duprat, 174 for Reichardt), the refined measures (facet-interior nodes, averaged normal, net flux) up to $t = 0.3$ (40 steps). On the flat top wall $|\mathbf{u} \cdot \mathbf{n}| \leq 1.5 \times 10^{-14}$.

Interpretation. The constraint is satisfied to round-off everywhere. The $\approx 1\,\%$ residual with per-element normals is a measurement artefact: on a curved wall, neighbouring elements meet with a slight kink (continuous geometry, discontinuous slope), so their face normals differ at shared edge nodes. The small net flux has the same origin, since the surface integral uses face normals. It does not grow systematically and should decrease at higher polynomial order. The behaviour is the same for both models, so Duprat's pressure-gradient feedback plays no part.

Related detail. At edge nodes the wall models distribute τ_w along the facet tangent of one element rather than the averaged tangent. Any normal part is removed by the constraint (a $1\,\%$ effect, edge nodes only). Using the projector's averaged normals would be cleaner, but this belongs to Neko's wall-model base class.

8 Multi-rank failure on Fritz: `Invalid face neigh.` (PR #7)

Symptom. The periodic hill ran on a desktop (gfortran, OpenMPI 5, 14 ranks). On Fritz (NHR@FAU; Intel compiler, OpenMPI 5 with UCX) every run on more than one rank stopped during the mesh connectivity setup with `*** ERROR: Invalid face neigh. ***`. The plain `mesh_checker` failed in the same way on 14 and 72 ranks, so neither the case file nor the wall model was involved. A serial run worked, and the mesh was identical on both machines (same element, point, face and edge counts).

Cause. `mesh_generate_external_facet_conn` and `mesh_generate_facet_numbering` in `src/mesh/mesh.f90` passed the result of the stack's `%array()` function directly as the buffer of the nonblocking `MPI_Issend`. A compiler may pass a function result through a temporary copy that is freed right after the call, while the synchronous send is still pending. The receiving rank then gets corrupted facet data, e.g. a neighbour element 0, and the consistency check of the facet numbering stops the run. `mesh_generate_edge_conn` in the same file already avoids this pattern, with the comment that it "breaks horribly on Cray".

Fix. Both sends now pass the stack's own storage through `select type`, as the edge exchange does. No other nonblocking MPI call in `src` passes a function result as its buffer. With the fix, `mesh_checker` on 72 ranks completed on Fritz. In the container (gfortran, OpenMPI 4.1.6), `mesh_checker` completed on 2 to 144 ranks for three periodic box meshes, and the forces of the periodic hill with and without the fix differ only at the level of run-to-run variation. Merged into `feature/wmles` as PR #7. The bug was also present on upstream `develop`; the same change, prepared on the branch `fix/mesh-issend-buffer`, is merged there as ExtremeFLOW/neko#2867.

9 Periodic hill at $Re_h = 10\,595$ with the Reichardt model

9.1 Case and runs

- **Geometry and mesh:** hill geometry of Breuer et al. as in the ERCOFTAC case, $L_x = 9h$, $L_y = 3.035h$, $L_z = 4.5h$. The `genmeshbox` mesh has $22 \times 8 \times 19$ elements, periodic in x and z; the user file maps it to the hill. With $N = 8$ this gives 1.74 million unique points, about $176 \times 64 \times 152$. There is only weak stretching towards the walls: 8 elements over the channel height.
- **Flow:** $Re_h = 10\,595$. `flow_rate_force` sets the bulk velocity at the hill crest to $U_b = 1$; the value $0.720958 = 2.035/H_{\text{mean}}$ was checked against the mean channel height. This is the definition used by Gloerfelt & Cinnella [3]; the evaluation confirms $U_b = 0.9999$.
- **Models and numerics:** Vreman ($c = 0.07$), gradient jump penalty (0.8, as a source term), `coupled_cg` for the velocity, variable Δt at CFL 0.5.
- **Wall model:** Reichardt on both walls ($\kappa = 0.41$, $C = 7.8$, $B_1 = 11$, $B_2 = 3$), sampling at GLL index 3. On the hill wall this gives $h_s = 0.062 \ldots 0.109\,h$.
- **Fixes to the student's case:** two settings in it were silently ignored by the current Neko:
  - the old `gradient_jump_penalty` block, so the runs would have had no GJP;
  - the Vreman keys `c_s` and `field_name`.

  Both were corrected (`periodic_hill/CHANGES.md`).
- **Output:**
  - `force_torque` on both walls every 10 steps, scaled with $2/(\rho U_b^2 L_x L_z)$. The scaled pressure and viscous parts are then the drag and skin-friction coefficients.
  - `fluid_stats` averaged over z (`"avg_direction": "z"`), sampled every 10 steps from $t = 150$ and written every 50 time units.

Two runs with the same case file reached $t = 600$:

Run	Set-up
Fritz (CPU)	Intel build with PR #7, OpenMPI 5, 72 ranks (one node). Restarted at $t = 173.6$. After a restart Neko's statistics start again at the restart time, so the statistics cover $t = 173.6 \ldots 600$, 47 flow-through times L_x/U_b.
Alex (GPU)	GNU build, CUDA, one GPU. Restarted at $t = 100$; statistics $t = 150 \ldots 600$, 50 flow-through times.

For comparison, Gloerfelt & Cinnella averaged over 55–80 flow-through times after 23 for the transient.

9.2 Evaluation

Two scripts in `periodic_hill/analysis` do the evaluation. `ph_stats.py` reads the 2D statistics files and combines the output windows weighted with their lengths. The Reynolds stresses are formed from the combined raw moments. The script interpolates the profiles spectrally inside the elements at $x/h = 0.05, 0.5, 1, \ldots, 8$; the interpolation was checked to round-off on the coordinates. `ph_forces.py` evaluates the forces, the forcing and the time stepping from the logs. Uncertainties are the standard errors of the means over windows of 50 time units. These windows are not fully independent, so the uncertainties are lower bounds.

The skin friction is not part of the statistics. The script reconstructs it the way the wall model computes τ_w: the mean velocity tangential to the wall is taken at the sampling point, and Reichardt's law is solved for u_τ. This gives $\tau_w(\langle u \rangle)$, not $\langle \tau_w \rangle$. Its magnitude is therefore approximate: the mean C_f obtained this way is 8 % (top wall) to 17 % (hill wall) below the `force_torque` value (Table 2). Its sign, and with it the separation and reattachment points, follows the mean flow.

The reference is the LES database of Gloerfelt & Cinnella [3] at $Re_h = 10\,595$: C_f on four grids from $64 \times 33 \times 32$ to $512 \times 256 \times 256$, and profiles. The experimental reattachment point of Rapp & Manhart [4] is $x/h = 4.21$.

9.3 Forces and momentum balance

Table 2: Time-averaged forces, scaled with $2/(\rho U_b^2 L_x L_z)$, and the flow-rate forcing f. Full averaging range of each run; ± standard error over windows of 50 time units.

	Fritz (CPU), $t \geq 173.6$	Alex (GPU), $t \geq 150$
hill wall, pressure drag $F_{x,p}$	0.0434 ± 0.0009	0.0449 ± 0.0005
hill wall, skin friction $F_{x,v}$	0.00202 ± 0.00006	0.00212 ± 0.00003
top wall, skin friction $F_{x,v}$	0.00616 ± 0.00006	0.00605 ± 0.00004
reconstructed mean C_f, hill / top wall	0.00169 / 0.00566	0.00176 / 0.00555
forcing $\langle f \rangle$	0.00906	0.00932
balance $\langle F_{x,3} + F_{x,4} \rangle / (2 H_{\mathrm{mean}} \langle f \rangle)$	1.009	1.008

The pressure drag dominates the force on the hill wall; its skin friction is only about 5 % of the total. The momentum balance closes to within 1 % in both runs. The remainder reflects that `force_torque` evaluates the resolved wall gradient, while the wall model imposes the traction only weakly. For comparison, the mean C_f of the hill wall in the reference is 0.00132 on the finest grid and 0.0007–0.0011 on the coarser ones. Figure 1 shows the time series.

9.4 Skin friction, separation and reattachment

Table 3: Zero crossings of C_f on the hill wall. Neko: reconstructed C_f, full averaging range. The reference values are from the C_f data of the database.

	separation	reattachment	secondary bubble	$C_{f,\max}$ (windward)
Neko Reichardt, Fritz (CPU)	0.55	2.78	–	0.011
Neko Reichardt, Alex (GPU)	0.55	2.71	–	0.012
Gloerfelt $512 \times 256 \times 256$	−0.12	4.31	6.74–7.08	0.033
Gloerfelt $256 \times 128 \times 128$	−0.16	4.32	6.78–7.02	0.030
Gloerfelt $128 \times 64 \times 64$	−0.11	4.49	–	0.023
Gloerfelt $64 \times 33 \times 32$	0.24	4.31	–	0.015
Experiment (Rapp & Manhart)		4.21		

The recirculation zone is much too short (Table 3, Figure 2). The flow stays attached over the crest with $C_f \approx 0.009 \ldots 0.010$, separates only at $x/h \approx 0.55$ and reattaches already at $x/h \approx 2.7$–2.8, against 4.3–4.5 in all reference grids. Over the reattachment points of the individual windows the scatter is about ±0.2 (Fritz) and ±0.1 (Alex), so the difference is not statistical. Inside the bubble, C_f only reaches about -0.0007 against $-0.002 \ldots -0.004$. The peak on the windward face, 0.011–0.012, is close to that of the coarsest reference grid, but the reconstruction from the mean velocity underestimates peaks.

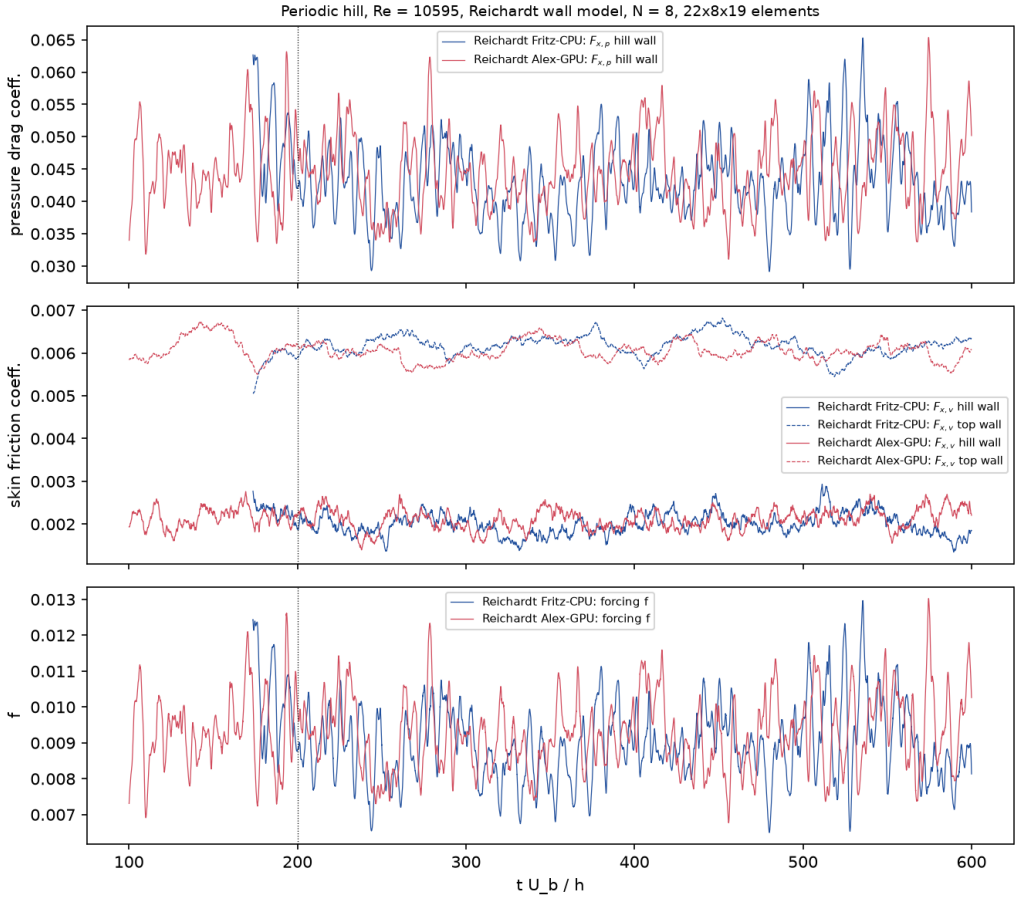


Figure 1: Forces and forcing over time, Reichardt model, Fritz (CPU) and Alex (GPU). The dotted line marks $t = 200$, the start of the common averaging range used in Section 9.7.

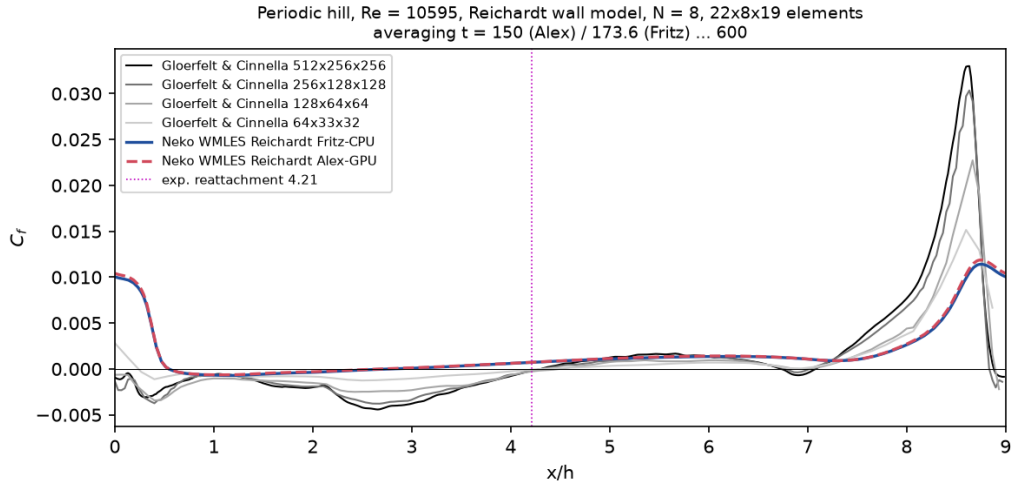


Figure 2: Skin friction on the hill wall: Neko WMLES with the Reichardt model (reconstructed from the mean velocity at the sampling point) and the LES of Gloerfelt & Cinnella [3] on four grids.

9.5 Profiles

Figures 3–4 compare the profiles with the finest reference grid; Table 4 lists characteristic values. At the wall, the Neko profiles show the slip velocity that the wall model allows.

- **Upper channel, and downstream from $x/h \approx 5$:** the mean velocity agrees well with the reference.
- **Bubble:** it starts with backflow of the right strength, $\langle u \rangle_{\min} = -0.17$ at $x/h = 1$ and -0.21 at $x/h = 2$, against -0.13 and -0.25 in the reference. It then closes early: at $x/h = 3$ the backflow is almost gone (-0.02 against -0.14), and at $x/h = 4$ the flow near the wall is forward.
- **Turbulence:** in the early shear layer at $x/h = 1$ and 2, $\langle u'u' \rangle$ and $-\langle u'v' \rangle$ are 15–35 % higher than in the reference; at $x/h = 0.5$ the excess is 0–15 %. Downstream of reattachment ($x/h \geq 4$) they are up to 15 % lower.
- **Crest ($x/h = 0.05$):** the fine reference grids show the shear layer starting at the wall, with $\langle u'u' \rangle$ up to 0.12. In the Neko runs the flow there is still attached, and the near-wall fluctuations stay at 0.03.

Table 4: Minimum of $\langle u \rangle$ and maxima of the Reynolds stresses along the stations. Neko: Fritz / Alex, full averaging range. Reference: Gloerfelt & Cinnella, $512 \times 256 \times 256$.

x/h	$\langle u \rangle_{\min}$		$\langle u'u' \rangle_{\max}$		$-\langle u'v' \rangle_{\max}$	
	Neko	ref.	Neko	ref.	Neko	ref.
0.5	−0.067 / −0.064	−0.080	0.119 / 0.128	0.112	0.042 / 0.045	0.042
1	−0.174 / −0.168	−0.135	0.109 / 0.120	0.093	0.052 / 0.056	0.042
2	−0.213 / −0.194	−0.253	0.101 / 0.114	0.087	0.043 / 0.047	0.036
3	−0.025 / −0.013	−0.140	0.090 / 0.100	0.091	0.034 / 0.037	0.035
4	slip > 0	−0.005	0.071 / 0.078	0.079	0.026 / 0.027	0.030
6	slip > 0	0.000	0.048 / 0.050	0.055	0.017 / 0.017	0.020

9.6 Discussion

The main deficiency is the separation over the crest. Separation there is driven by the adverse pressure gradient on the lee side of the hill. Reichardt's law is an equilibrium law without a pressure-gradient term: as long as the velocity at the sampling point is directed downstream, the model returns a positive, here large, wall shear stress. The delayed separation at $x/h \approx 0.55$, the attached crest with $C_f \approx 0.009 \ldots 0.010$ and the low fluctuations at $x/h = 0.05$ are consistent with this. A shorter bubble follows. A second factor is resolution: with 8 elements over the height, the crest region and the early shear layer are coarse. Gloerfelt & Cinnella identify exactly this region as the most sensitive one for the whole solution.

Two next steps follow directly:

- The Duprat model in `local` mode, which is designed for pressure-gradient effects, on the same mesh and with the same evaluation. It was tested in the original paper [1] on this flow.
- A mesh with more elements, or stronger clustering, in the wall-normal direction near the crest, to separate the model effect from the resolution effect.

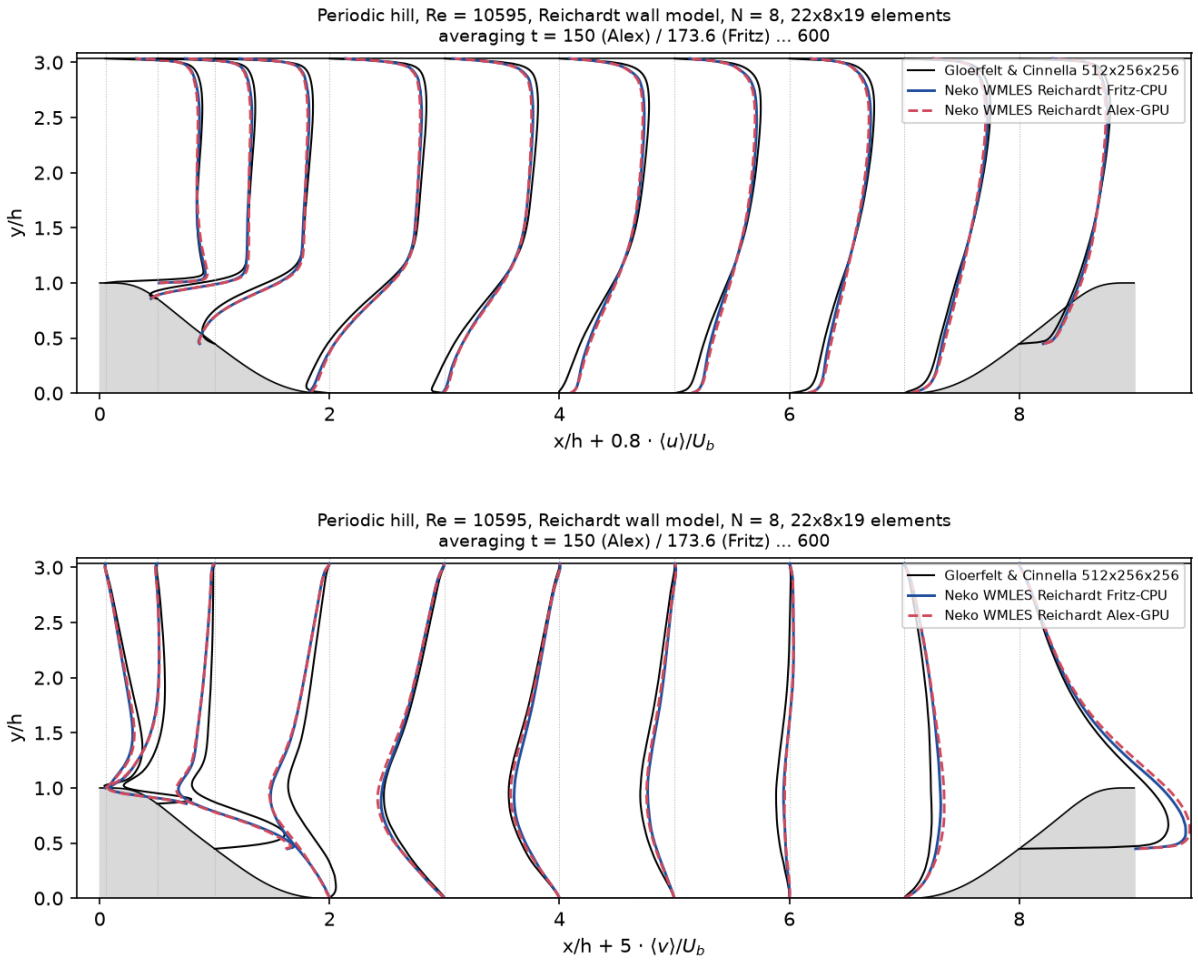


Figure 3: Mean velocity $\langle u \rangle$ (top) and $\langle v \rangle$ (bottom), Reichardt model, against Gloerfelt & Cinnella [3], $512 \times 256 \times 256$.

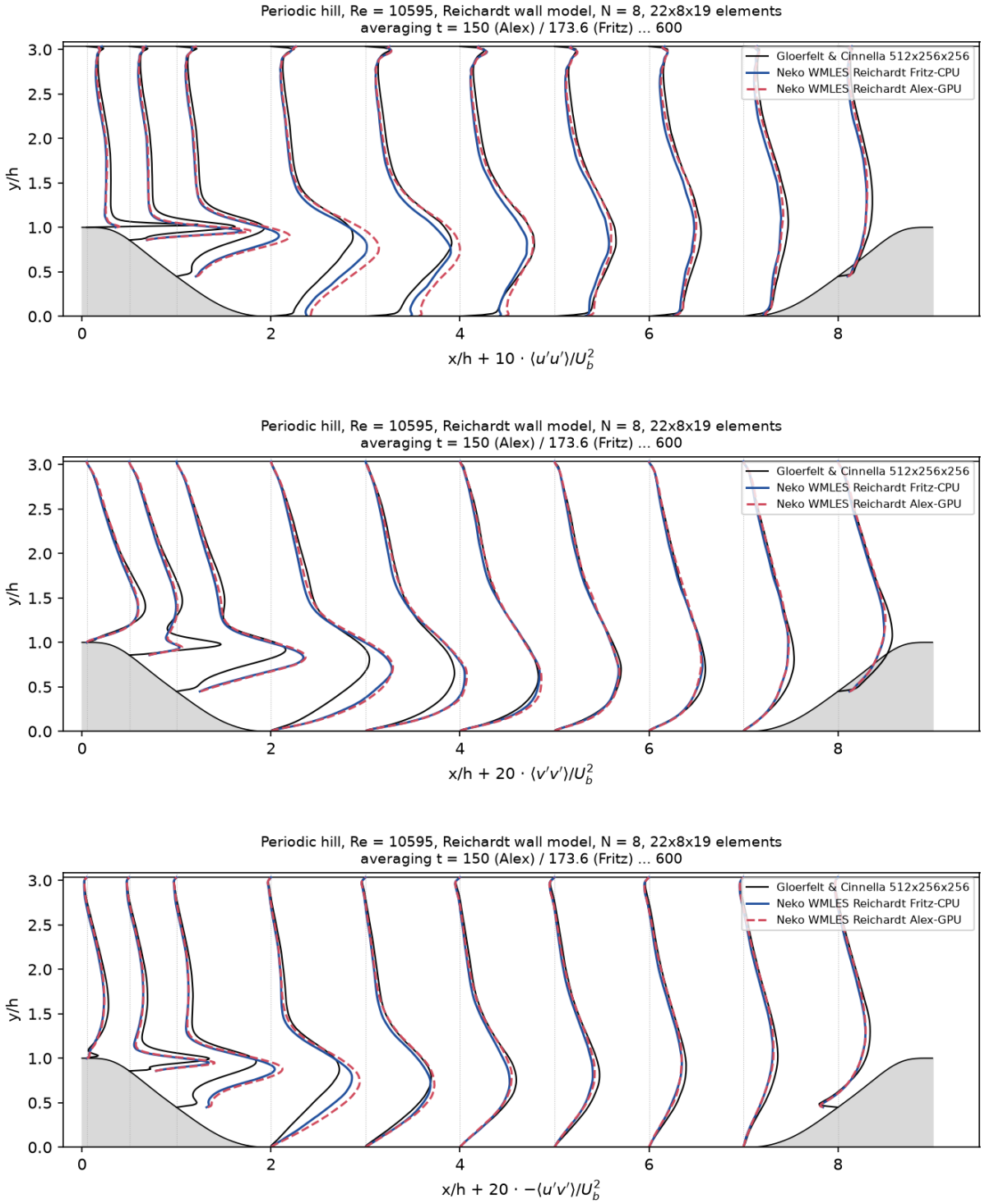


Figure 4: Reynolds stresses $\langle u'u' \rangle$, $\langle v'v' \rangle$ and $-\langle u'v' \rangle$, Reichardt model, against Gloerfelt & Cinnella [3], $512 \times 256 \times 256$.

9.7 GPU (Alex) versus CPU (Fritz)

For a like-for-like comparison, both runs were evaluated over the same windows, $t = 200 \ldots 600$: eight windows of 50 time units.

- **Agreement:** the mean velocity and C_f agree closely. The separation point is the same (0.547 against 0.546). The reattachment points agree within their scatter: 2.71 ± 0.03 on Alex and 2.78 ± 0.08 on Fritz, 0.9σ apart.
- **Forces:** they differ by 2–6 % at 1.6–2.3σ. On the hill wall, $F_{x,p}$ is +4.1 % and $F_{x,v}$ +5.5 % on the GPU; on the top wall, $F_{x,v}$ is −2.4 %. The forcing $\langle f \rangle$ is +3.5 %, at 1.8σ.
- **Turbulence in the shear layer:** here the GPU run is systematically higher. At $x/h = 0.5 \ldots 4$ the peaks of $\langle u'u' \rangle$ are 8–13 % higher, at 1.9–3.1σ, and those of $-\langle u'v' \rangle$ 5–10 % higher. 16 of 18 peak values of $\langle u'u' \rangle$, $\langle v'v' \rangle$ and $-\langle u'v' \rangle$ at $x/h = 0.5, 1, 2, 3, 4, 6$ are higher on the GPU. At $x/h = 6$ the difference has vanished (full table in `results_Re10595_reichardt/common/gpu_cpu_windows.txt`).

The consistent sign first suggested a systematic difference between the CUDA and the CPU code path in a device-specific part of the solver:

- the LES model;
- the gradient jump penalty;
- the dealiased advection.

The Duprat runs (Section 10.6) speak against this. They used the same two builds, so the same solver code on both machines, and show no such pattern. A solver-level code-path difference is therefore unlikely. Two explanations remain:

- a fluctuation that the window statistics underestimate. The 18 peak values come from the same eight windows and are correlated, so "16 of 18" overstates the significance;
- something specific to the Reichardt kernels, although the runs agree near the wall, where the wall model acts.

Statistics cannot settle this; a deterministic test can. Both machines restart from the same checkpoint with a fixed Δt and write the forces every step for about 20 steps. Identical algorithms give differences at round-off level, like a CPU run with a different rank count. A code-path difference shows from the first step on. The case file, a comparison script and instructions are in `periodic_hill/gpu_cpu_check`. The GPU warning was nevertheless removed after the Duprat comparison (Section 10.6); the test remains useful as a check.

An attempt to run the same comparison in the container with the OpenCL backend (PoCL on the CPU) did not get that far. The OpenCL run produced NaN in the first pressure solve, even without wall model, GJP and LES model. The OpenCL channel case had run before, so this concerns the PoCL test environment with this case, not CUDA, and was not pursued further.

9.8 Performance

On one Alex GPU a time step took 0.051 s (median), on a full Fritz node with 72 ranks 0.403 s, a factor of about 8. Both runs used $\Delta t \approx 3.35 \cdot 10^{-3}$ at a mean CFL of 0.48–0.49. By the total elapsed times in the logs, the GPU run needed 2.2 hours for 500 time units and the Fritz run 14.6 hours for 426 time units.

10 Periodic hill at $Re_h = 10\,595$ with the Duprat model

10.1 Case and runs

The case is the one of Section 9. Mesh, numerics, forcing and output are the same; only the wall model differs:

- **Wall model:** Duprat on both walls with $\kappa = 0.41$, $\beta = 0.78$ and $A = 17$, sampling at GLL index 3. The pressure gradient is evaluated locally (`"type": "local"`) with `filter_time` 1.0 and `start_time` 50.
- **Additional output:** `user_stats` of the registry fields `tau`, `duprat_dpds` and `duprat_alpha`, averaged over z from $t = 150$. This gives $\langle|\tau_w|\rangle$, the filtered $\partial p/\partial s$ seen by the model, and α along both walls.

Both runs reached $t = 600$. They used the same two builds as the Reichardt runs, so the solver code on each machine is the same for both models.

Run	Set-up
Fritz (CPU)	Intel build, 72 ranks. The first job reached the 24-hour limit at $t = 493.8$, after its last checkpoint at $t = 483.394$. The second job restarted from that checkpoint. The statistics therefore cover $t = 150 \ldots 450$ and $483.4 \ldots 600$, 46 flow-through times. The window $450 \ldots 483.4$ was lost because its output at $t = 500$ was never written. The logs of both jobs are joined at the restart. The filtered $\partial p/\partial s$ is not in the checkpoint and builds up again within about one `filter_time` after the restart.
Alex (GPU)	GNU build, CUDA, one GPU, from $t = 0$; statistics $t = 150 \ldots 600$, 50 flow-through times.

The evaluation uses the scripts and the C_f reconstruction of Section 9, now with the Duprat law and the mean filtered $\partial p/\partial s$. For the Fritz run, `ph_stats.py` takes one start time per output series (150 and 483.394), and `ph_forces.py` uses each log only up to the start of the next. The results are in `periodic_hill/analysis/results_Re10595_duprat`.

10.2 Forces

Table 5: Time-averaged forces, scaled with $2/(\rho U_b^2 L_x L_z)$, and the flow-rate forcing f for both models. Full averaging range of each run; ± standard error over windows of 50 time units. The Reichardt columns repeat Table 2.

	Duprat		Reichardt	
	Fritz (CPU)	Alex (GPU)	Fritz (CPU)	Alex (GPU)
hill wall, $F_{x,p}$	0.0622 ± 0.0009	0.0595 ± 0.0012	0.0434 ± 0.0009	0.0449 ± 0.0005
hill wall, $F_{x,v}$	0.00285 ± 0.00009	0.00286 ± 0.00012	0.00202 ± 0.00006	0.00212 ± 0.00003
top wall, $F_{x,v}$	0.00854 ± 0.00006	0.00859 ± 0.00006	0.00616 ± 0.00006	0.00605 ± 0.00004
mean C_f from $\langle\|\tau_w\|\rangle$, hill/top	0.00284 / 0.00803	0.00284 / 0.00805	–	–
reconstr. mean C_f, hill/top	0.00236 / 0.00774	0.00236 / 0.00778	0.00169 / 0.00566	0.00176 / 0.00555
forcing $\langle f \rangle$	0.01291	0.01243	0.00906	0.00932
balance	1.010	1.010	1.009	1.008

- **Compared with Reichardt** (same machine): the pressure drag on the hill is 33–43 % higher, and the skin friction on both walls 35–42 % higher. The forcing that keeps $U_b = 1$ rises by 33–42 %. The momentum balance again closes to within 1 %.
- **Wall shear stress statistic:** the mean of the model's $|\tau_w|$, with the sign of the mean flow, matches the `force_torque` skin friction of the hill wall to 0.4 % (0.00284 against 0.00285). On the top wall it is 6 % lower.

- **Law against flow:** the source of the higher friction can be separated by applying both laws to the same mean flow of the Duprat runs:
  - On the hill wall the Duprat law gives twice the mean C_f of the Reichardt law (0.00236 against 0.00113). Most of the difference comes from the windward face.
  - On the top wall the Duprat law gives 25 % more (0.0077 against 0.0062), although α is at least 0.91 there (mean 0.95). The remaining 12 % come from the flow: the velocity at the top sampling points is about 7 % higher than in the Reichardt runs.
- **Against the reference:** the mean C_f of the hill wall, 0.0024–0.0029, is about twice that of the finest grid of Gloerfelt & Cinnella (0.00132). With Reichardt it was 0.0017–0.0021.

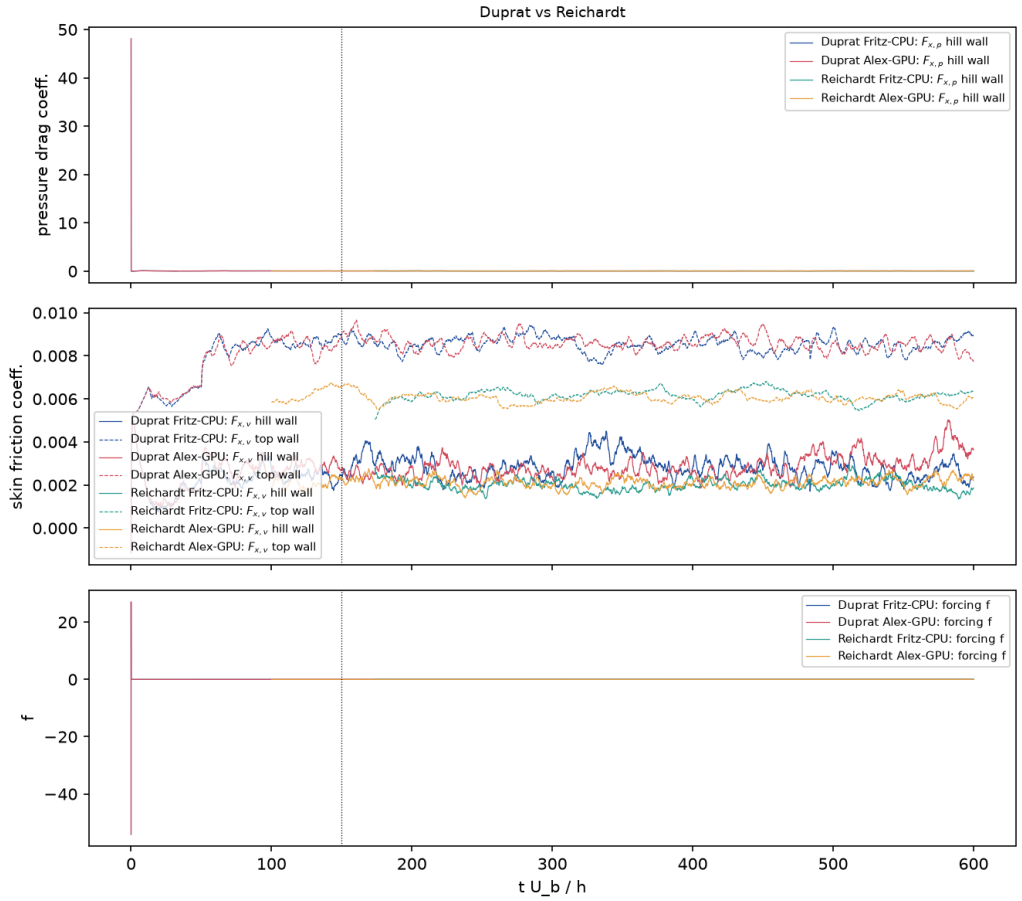


Figure 5: Forces and forcing over time for both models, Fritz (CPU) and Alex (GPU).

10.3 Skin friction, separation and reattachment

- **Reattachment:** it moves downstream by about 0.5, from 2.7–2.8 to 3.2–3.4. That is still about 1 short of the reference.
- **Separation:** it moves upstream from 0.55 to 0.45. The flow is still attached over the crest, where the reference separates at or just before it, but C_f there drops from 0.010 to 0.007.
- **Bubble:** it is twice as strong as with Reichardt. The reconstruction gives $C_f \approx -0.0014$ and $\langle|\tau_w|\rangle$ gives -0.0028, against $-0.0038 \ldots -0.0044$ in the reference.

Table 6: Zero crossings and extrema of C_f on the hill wall. Neko: reconstructed C_f; in parentheses, the mean reattachment point over the output windows with its standard error, and the bubble minimum from $\langle|\tau_w|\rangle$. Reference values as in Table 3.

	separation	reattachment	$C_{f,\min}$ (bubble)	$C_{f,\max}$ (windward)
Neko Duprat, Fritz (CPU)	0.45	3.33 (3.37 ± 0.10)	-0.0015 (-0.0028)	0.031
Neko Duprat, Alex (GPU)	0.45	3.21 (3.21 ± 0.08)	-0.0013 (-0.0029)	0.031
Neko Reichardt, Fritz (CPU)	0.55	2.78 (2.78 ± 0.07)	-0.0007	0.011
Neko Reichardt, Alex (GPU)	0.55	2.71 (2.71 ± 0.03)	-0.0006	0.012
Gloerfelt $512 \times 256 \times 256$	-0.12	4.31	-0.0044	0.033
Gloerfelt $256 \times 128 \times 128$	-0.16	4.32	-0.0038	0.030
Experiment (Rapp & Manhart)		4.21		

- **Windward peak:** $C_{f,\max} = 0.031$ at $x/h = 8.63$ now matches the fine reference grids (0.030–0.033). Reichardt gave 0.011. Here the strong favourable gradient on the windward face acts as the model intends.
- **Recovery region** ($x/h = 4 \ldots 7$): C_f from $\langle|\tau_w|\rangle$ is about 0.002 and the reconstruction gives 0.0007, against 0.0008 on average in the reference.

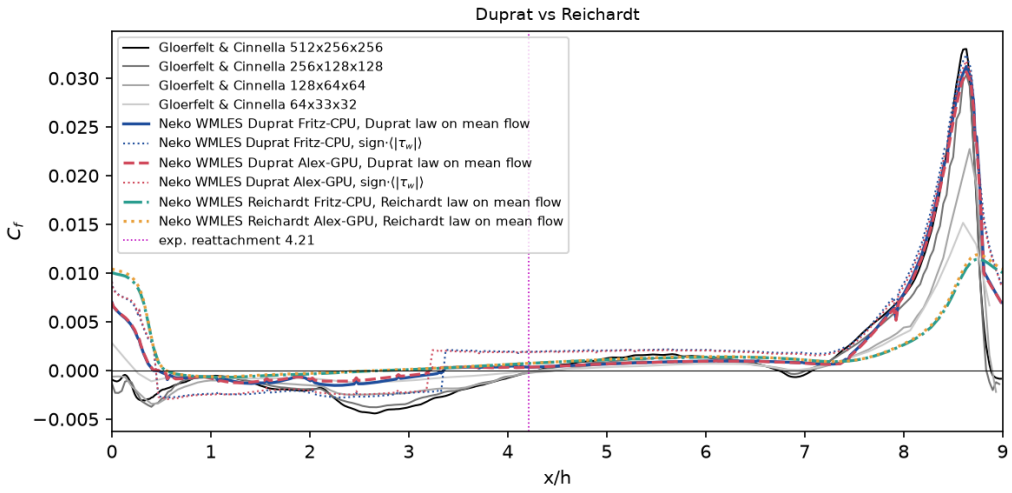


Figure 6: Skin friction on the hill wall: Neko WMLES with the Duprat model (reconstructed with the Duprat law, solid and dashed; from $\langle|\tau_w|\rangle$ with the sign of the mean flow, dotted) and with the Reichardt model, against Gloerfelt & Cinnella [3].

10.4 Profiles

- **Mean velocity** (Table 7, Figure 7):
  - The largest gain is at $x/h = 3$: the backflow is -0.08 to -0.12, against -0.14 in the reference and only -0.01 to -0.03 with Reichardt.
  - At $x/h = 2$ the backflow is close to the reference on the CPU (-0.24 against -0.25) and 20 % weaker on the GPU.
  - At $x/h = 0.5$ it is too strong, -0.11 against -0.08.
- **Turbulence:**
  - In the shear layer at $x/h = 1$ and 2, the peaks of $\langle u'u' \rangle$ and $-\langle u'v' \rangle$ are 15–30 % above the reference, as with Reichardt.

Table 7: Minimum of $\langle u \rangle$ and maxima of the Reynolds stresses along the stations. Neko: Fritz / Alex, full averaging range. Reference: Gloerfelt & Cinnella, $512 \times 256 \times 256$. "slip": the minimum is the slip velocity at the wall, positive.

x/h	$\langle u \rangle_{\min}$			$\langle u'u' \rangle_{\max}$		$-\langle u'v' \rangle_{\max}$	
	Duprat	Reichardt	ref.	Duprat	ref.	Duprat	ref.
0.5	−0.112 / −0.111	−0.067 / −0.064	−0.080	0.129 / 0.127	0.112	0.046 / 0.047	0.042
1	−0.149 / −0.137	−0.174 / −0.168	−0.135	0.110 / 0.108	0.093	0.051 / 0.052	0.042
2	−0.237 / −0.197	−0.213 / −0.194	−0.253	0.104 / 0.103	0.087	0.046 / 0.044	0.036
3	−0.115 / −0.084	−0.025 / −0.013	−0.140	0.100 / 0.099	0.091	0.041 / 0.039	0.035
4	slip	slip	−0.005	0.086 / 0.087	0.079	0.034 / 0.033	0.030
6	slip	slip	0.000	0.064 / 0.064	0.055	0.023 / 0.022	0.020

  - Downstream of reattachment ($x/h \geq 4$) they are now 9–16 % above the reference. With Reichardt they were up to 15 % below.

10.5 The pressure gradient seen by the model

Figure 8 shows the mean α and the mean filtered $\partial p/\partial s$ along the hill wall. Table 8 compares the filtered $\partial p/\partial s$ with the gradient of the mean pressure. Both are taken at the sampling points, and both are signed with respect to the local mean flow direction, so they are comparable.

Table 8: Range of the mean filtered $\partial p/\partial s$ used by the model and of the gradient of the mean pressure along the hill wall, at the sampling points. Positive values are adverse to the mean flow at the sampling point. Fritz (CPU); the Alex values agree to within a few per cent.

region (x/h)	model, $\langle \partial p/\partial s \rangle_{\text{filt}}$	mean pressure, $\partial \langle p \rangle / \partial s$
crest, 0–0.4	$+0.011 \ldots + 0.146$	$+0.014 \ldots + 0.248$
bubble, 0.5–3 (backflow)	$-0.012 \ldots + 0.004$	$-0.122 \ldots - 0.024$
recovery, 4–7	$-0.001 \ldots + 0.016$	$+0.035 \ldots + 0.061$
windward face, 8.3–8.8	$-0.578 \ldots - 0.143$	$-0.858 \ldots - 0.200$

- **Strength:** averaged over each region, the model sees about two thirds of the mean gradient on the windward face and a little more than half at the crest. In the bubble it sees only 1/13 of it, and in the recovery region 1/8 to 1/9. CPU and GPU agree.
- **Cause:** the gradient is projected on the *instantaneous* wall-parallel velocity and limited to the Stokes bound before it is filtered (Section 5.6). Where the flow direction changes often, as in the bubble and near reattachment, the projections cancel in the time filter, and the limit cuts the peaks.
- **Element boundaries:** α shows narrow spikes there, and $|\partial p/\partial s|$ dips to about half of its neighbours. They appear identically on CPU and GPU, so they are not a device effect. The most likely cause is the one-sided pressure gradient at the element faces together with the Stokes limit.

10.6 GPU (Alex) versus CPU (Fritz)

The comparison uses all output windows of 50 time units: eight on Fritz and nine on Alex. The table is in `results_Re10595_duprat/gpu_cpu_windows_duprat.txt`, made with the new script `gpu_cpu_windows.py`. Run on the Reichardt data, the script reproduces the table of Section 9.7 exactly.

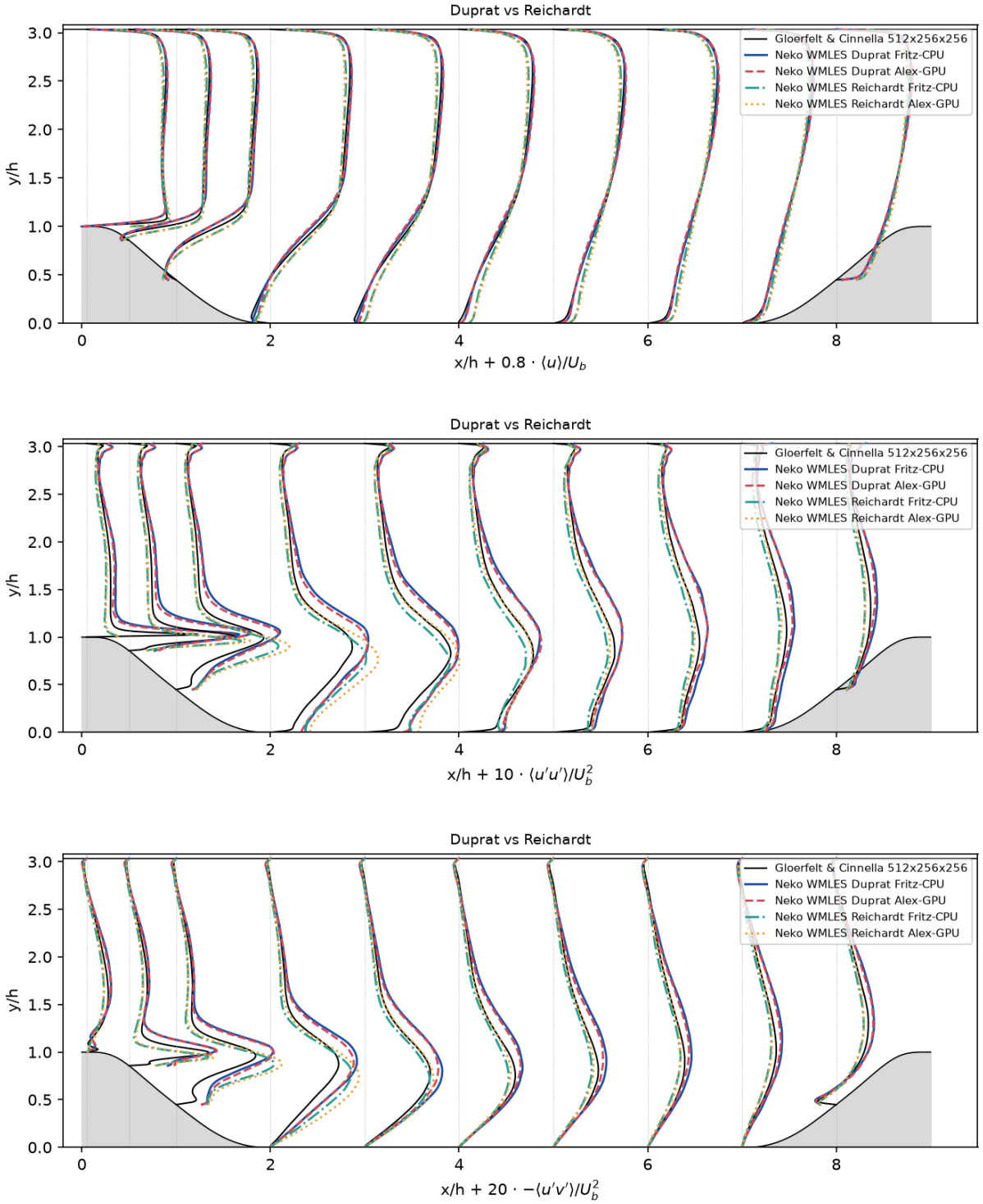


Figure 7: Mean velocity $\langle u \rangle$, $\langle u'u' \rangle$ and $-\langle u'v' \rangle$ for both models and both machines, against Gloerfelt & Cinnella [3], $512 \times 256 \times 256$.

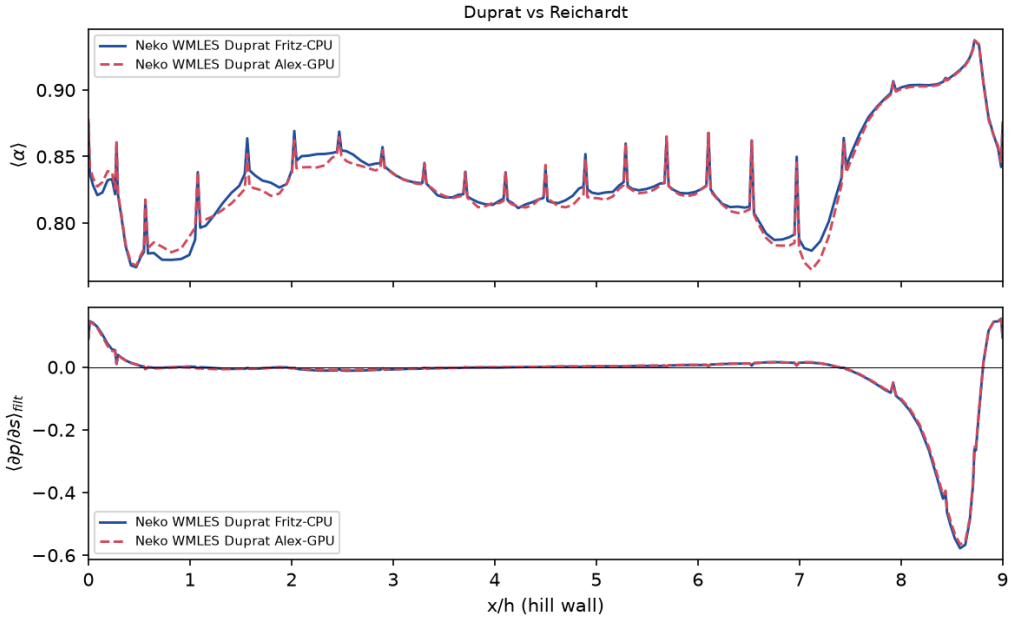


Figure 8: Mean $\alpha = u_\tau^2/u_{\tau p}^2$ (top) and mean filtered $\partial p/\partial s$ (bottom) along the hill wall, Duprat model, Fritz (CPU) and Alex (GPU).

- **Mean flow and skin friction:**
  - The separation point is the same, 0.446 against 0.451.
  - The reattachment points over the windows are 3.37 ± 0.10 on the CPU and 3.21 ± 0.08 on the GPU, 1.2σ apart.
  - C_f, α and $\partial p/\partial s$ lie on top of each other (Figures 6 and 8).
- **Forces:**
  - The skin friction agrees within 0.6 % on both walls.
  - The pressure drag is 4.5 % and the forcing 3.8 % lower on the GPU, at 1.7–1.8σ. With Reichardt, both were about 4 % *higher* on the GPU, at similar significance.
- **Turbulence:** no systematic difference. Only 4 of the 18 peak values are higher on the GPU. The differences are −5.5 to +2.6 %, all within 1.3σ.
- **Largest difference:** the backflow at $x/h = 2$, −0.240 on the CPU against −0.199 on the GPU (3.4σ). The CPU bubble is slightly stronger, which fits its later reattachment.

Two conclusions follow:

- **No systematic difference:** there is no sign of a systematic difference between the CUDA and the CPU code path. Both models used the same builds on each machine, so this also argues against a solver-level cause of the Reichardt observation (Section 9.7).
- **Uncertainties too small:** both comparisons show 2–3σ differences, but in different quantities and with different signs. Standard errors over 50-unit windows therefore underestimate the run-to-run scatter of this flow, presumably because of slow modulations of the bubble.

On this basis the GPU warnings of both models were removed. The deterministic restart test of Section 9.7 remains the clean way to confirm that the code paths agree.

10.7 Performance

- **Cost per step:** 0.551 s on Fritz and 0.061 s on Alex (median). That is 37 % and 20 % more than with Reichardt.
- **Time step:** Δt is 3–6 % smaller, $3.16 \cdot 10^{-3}$ and $3.23 \cdot 10^{-3}$.
- **Elapsed time:** the Fritz jobs took 23.9 and 5.7 hours, about 174 s per time unit against 123 s with Reichardt. The Alex run took 3.2 hours for 600 time units.
- **Likely cause** (not profiled): U^* is evaluated by quadrature inside the Newton iteration, and only on the ranks that own wall elements. On the CPU this also unbalances the load.

10.8 Discussion

The Duprat model improves the hill flow where the pressure gradient is strong and acts as the model intends:

- the windward skin-friction peak;
- the earlier separation;
- a stronger and longer bubble, with reattachment 0.5 further downstream.

The reattachment is still about 1 short of the reference, and the friction is too high.

The analysis of the pressure gradient explains much of this. In the bubble and in the recovery region, the model sees only a fraction of the mean pressure gradient (Section 10.5). It is therefore closer to an equilibrium model there than its formulation intends. Three changes to the local mode follow; all keep the interface:

1. Filter the gradient *vector* first, then project it on the filtered flow direction and apply the limit. This avoids the cancellation in reversing flow.
2. Average ∇p continuously across element faces before sampling. This removes the dips at the element boundaries.
3. Include the driving body force in the pressure gradient. On the hill it is $f \approx 0.012$: small on the windward face, but 20–35 % of the mean gradient in the recovery region.

These changes can be tested on this case with the same evaluation. Independently of the model, the coarse wall-normal resolution near the crest (Section 9) remains a factor.

11 Test environment

- Cloud container: gfortran, MPICH, json-fortran 9.0.3 (built locally), Neko built with `-O1` on CPU and with `-with-opencl`. PoCL 5.0 (OpenCL on the CPU), with Neko's device selection patched from `CL_DEVICE_TYPE_GPU` to `CL_DEVICE_TYPE_ALL` in the test copy only.
- No `nvcc`, `hipcc`, Metal compiler, or GPU hardware.
- In this container, `mpirun -np 4` with MPICH started independent single-rank copies (Ubuntu's MPICH is built for PMIx, which Hydra does not serve), so the model comparisons used one rank. For the multi-rank tests of Section 8, OpenMPI 4.1.6 was installed and Neko built against it.
- With PoCL, the periodic hill produced NaN in the first pressure solve, even without wall model, GJP and LES model (Section 9.7).

- pFUnit 4.15.0 was built in the container. The unit suites of the upstream branches were run with gfortran 13.3 and OpenMPI 4.1.6, with the compiler flags of the upstream CI (`-g -O2 -pedantic -std=f2008`), in single and double precision.
- The ad-hoc test harnesses (kernel drivers, SciPy references, user files) lived in the session's scratch space and are not part of the repository. The kernel tests now live on as pFUnit suites; see the open items.

12 Open items and recommendations

1. **GPU validation:** both models have run the full hill on Alex (CUDA). Mean flow and skin friction agree with the CPU runs.
   - The higher shear-layer turbulence of the Reichardt GPU run (Section 9.7) does not reappear with Duprat (Section 10.6).
   - The GPU warnings have been removed. The deterministic restart test in `periodic_hill/gpu_cpu_check` would still settle the Reichardt observation.
2. **Duprat `local` mode:** the hill campaign is done (Section 10). The model passes on too little of the pressure gradient where the flow reverses. Implement and test, on the same case with the same evaluation:
   - filtering the gradient vector before the projection and the limit;
   - averaging ∇p continuously across element faces;
   - including the driving body force.

   Also look at the cost: 37 % more per step than Reichardt on the CPU.
3. **Wall-normal resolution of the hill mesh:** 8 elements over the height. A finer or more strongly clustered mesh near the crest would separate the wall-model effect from the resolution effect.
4. **Mean wall shear stress as a statistic:** the Duprat runs average $|\tau_w|$ through `user_stats`. On the hill wall this matches the `force_torque` skin friction to 0.4 %. A signed streamwise τ_w would also resolve the bubble, where the mean of the magnitude overestimates $|C_f|$.
5. **Thesis results:** confirm with D. Bole which code version (with or without FIX 10) produced the hill results. Re-examine the Duprat hill results and the `h_index` = 7 channel result in light of Sections 5.3 and 5.4.
6. **Filter state in checkpoints:** the filtered $\partial p/\partial s$ restarts from zero after a restart; it could be added to the checkpoint.
7. **Normals at edge nodes:** let the wall-model base class use the projector's averaged normals (Section 7).
8. **Permanent tests:** the upstream branches now carry pFUnit suites for the CPU kernels:
   - Reichardt: 5 tests;
   - Duprat: 8 tests, including the filter update.

   Both pass in single and double precision with the CI flags, and so does the full unit suite. An integration test is still missing.

9. **Documentation:** the user guide stated that `shear_stress` and `wall_model` only work with axis-aligned boundaries. In current Neko both require `full_stress_formulation`, and the coupled projector constrains the normal velocity along the resolved node normal, as the hill tests confirm. The Duprat branch corrects the user guide and the comment in `wall_model_bc.f90`.

10. **Upstream:** both models are on branches based on ExtremeFLOW/neko `develop`, `feature/reichardt-wall-model` and `feature/duprat-wall-model`. The Reichardt PR is open. The two PRs register their model in the same lists, so the second one to be merged needs a merge of `develop`, and `WALLM_KNOWN_TYPES` then has 7 entries.

A Case-file examples

Reichardt (constants optional, shown with defaults):

```
{ "type": "wall_model", "zone_indices": [3, 4], "model": "reichardt",
  "kappa": 0.41, "C": 7.8, "B1": 11.0, "B2": 3.0,
  "sampling": { "type": "gll", "value": 2 } }
```

Duprat, constant pressure gradient:

```
{ "type": "wall_model", "zone_indices": [3, 4], "model": "duprat",
  "kappa": 0.41,
  "pressure_gradient": { "type": "constant", "value": 0.0 },
  "sampling": { "type": "gll", "value": 3 } }
```

Duprat, local pressure gradient, with diagnostics output:

```
{ "type": "wall_model", "zone_indices": [3, 4], "model": "duprat",
  "kappa": 0.41, "beta": 0.78, "A": 17.0,
  "pressure_gradient": { "type": "local", "filter_time": 1.0,
                         "start_time": 100.0 },
  "sampling": { "type": "gll", "value": 3 } }

{ "type": "field_writer", "fields": ["duprat_dpds", "duprat_alpha"],
  "output_filename": "duprat_wall",
  "output_control": "simulationtime", "output_value": 1.0 }
```

All wall models need `"full_stress_formulation": true` and a coupled velocity solver, e.g. `"type": "coupled_cg"`.

B Mapping of the student's case-file keys

Student (`curved_neko`)	`feature/wmles`
`h_index`	`sampling: {"type": "gll", "value": ...}` (legacy key still accepted)
`use_constant_dpdx: true`	`pressure_gradient: {"type": "constant", ...}`
`dpdx_constant`	`value` (note: u_p without the factor 1/2)
`use_constant_dpdx: false`	`pressure_gradient: {"type": "local", ...}`
`t_filter`	`filter_time` (now required in `local`)
`t_filter_start`	`start_time` (default 0)
`kappa`, `beta`, `A`	unchanged (`kappa` required)
Reichardt `B` (unused)	removed; `C`, `B1`, `B2` added

C Files added or changed

New (under `src/wall_models/`):

- `reichardt.f90`, `duprat.f90`;
- `bcknd/cpu/reichardt_cpu.f90`, `bcknd/cpu/duprat_cpu.f90`;
- `bcknd/device/reichardt_device.F90`, `bcknd/device/duprat_device.F90`;
- for each backend directory `bcknd/device/{cuda,hip,opencl,metal}/`, a launcher and a kernel file per model:
  - `reichardt.{cu,hip,c,m}` and `reichardt_kernel.{h,cl,metal}`;
  - `duprat.{cu,hip,c,m}` and `duprat_kernel.{h,cl,metal}`.

Changed:

- `src/wall_models/wall_model_fctry.f90`;
- `src/Makefile.am`, `src/.depends`, `src/.depends_device`;
- `src/device/opencl/prgm_lib.F90`, `prgm_lib.h`;
- `doc/schemas/fluid-pnpn-boundary-conditions.schema.json`;
- `doc/pages/user-guide/case-file.md`;
- `CHANGELOG.md`.

References

[1] C. Duprat, G. Balarac, O. Métais, P. M. Congedo, O. Brugière, A wall-layer model for large-eddy simulations of turbulent flows with/out pressure gradient, *Physics of Fluids* 23, 015101 (2011), doi:10.1063/1.3529358.

[2] H. Reichardt, Vollständige Darstellung der turbulenten Geschwindigkeitsverteilung in glatten Leitungen, *ZAMM* 31(7), 208–219 (1951), doi:10.1002/zamm.19510310704.

[3] X. Gloerfelt, P. Cinnella, Large eddy simulation requirements for the flow over periodic hills, *Flow, Turbulence and Combustion* 103(1), 55–91 (2019), doi:10.1007/s10494-018-0005-5.

[4] C. Rapp, M. Manhart, Flow over periodic hills: an experimental study, *Experiments in Fluids* 51, 247–269 (2011).

[5] D. Bole, Master's thesis on wall-modelled LES with the Reichardt and Duprat wall models in Neko (internal report, 2026).